Lecture 38: Deep Learning (Part-3) — Practice Problem Solutions (Hinglish)

Sabhi 15 practice problems ke complete mathematical derivations, code solutions aur explanations.

Solution 1: Regression vs Multi-Class Classification Architecture

- **Regression**: Output $y \in \mathbb{R}$ ek continuous number hota hai (jaise electrical power $420 - 495$ MW). Isme output layer mein **sirf 1 neuron** hota hai aur koi activation function nahi lagaya jata (identity linear output $z = Wx + b$). Agar ReLU lagate toh negative predictions ruk jate, aur Sigmoid lagate toh range sirf 0 se 1 ho jati. Loss function `nn.MSELoss()` hota hai.
- **Multi-Class Classification**: Target $y \in \{0, 1, \ldots, C-1\}$ discrete class hoti hai ($C = 7$ date fruits ke liye). Output layer mein $C = 7$ **neurons** hote hain, jo har class ka raw score (logit) emit karte hain. PyTorch mein model ke andar Softmax nahi lagate kyunki `nn.CrossEntropyLoss()` internally Softmax handle karta hai.

Solution 2: Feature Standardization Ki Zaroorat

Jab features ka scale alag-alag ho ($x_1 \approx 500,000$ aur $x_2 \approx 0.7$), toh weights ke gradients mein zameen-aasmaan ka farq aa jata hai:

$$\frac{\partial L}{\partial w_j} = \frac{\partial L}{\partial z} \cdot x_j$$

Isse loss function ka contour ek **bahut lamba, patla elliptical (oval) katora** ban jata hai. Ek constant learning rate lene par gradient descent steep direction mein bounce (oscillate) karta rehta hai aur flat direction mein aage nahi badhta. `StandardScaler` sabhi features ko $\mu = 0$ aur $\sigma = 1$ par set kar deta hai, jisse contours gol (spherical) ban jate hain aur gradient descent seedhe minimum tak pahunchta hai.

Solution 3: Data Leakage Feature Scaling Mein

- **Data Leakage** ka matlab hai testing data ki information training phase mein leak ho jana. Agar split karne se pehle poore dataset par `fit_transform` chalaya, toh scaler test set ke mean aur variance ko calculate kar lega:

$$\mu = \text{mean}(X_{\text{train}} + X_{\text{test}})$$

Isse training set pehle se hi test set ke distribution ko jaan leta hai, jo ek serious flaw hai.

- **Sahi tareeqa**:

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train) # Train se mu aur sigma ni
X_test_scaled = scaler.transform(X_test)       # Test par wahi mu aur sig
```

Solution 4: PyTorch Data Pipeline Mechanics

1. `TensorDataset` : Input features aur target labels ko index-wise baandh deta hai taaki `dataset[i]` se `(x_i, y_i)` mile.
2. `DataLoader` : Is dataset ko batches mein divide karta hai:
    - Mini-batching: $B = 32$ samples ka ek batch banata hai.
    - Shuffling: Har epoch ke baad data ko random mix karta hai.
    - Multi-threading data loading facilitate karta hai.
3. **Mini-batching kyun?** Poora data ek saath RAM/GPU mein bhejenge toh memory overflow ho jayegi. 32-32 ke batches lene se memory limit mein rehti hai aur gradient noise model ko local minima se bahar nikalne mein madad karta hai.

Solution 5: Tensor Data Types (`torch.float32` vs `torch.long`)

- **Regression (`nn.MSELoss`)**: Numbers ka difference square karta hai, isiliye target continuous float (`torch.float32`) hona chahiye.
- **Classification (`nn.CrossEntropyLoss`)**: PyTorch internally target class label ko array ke index ki tarah use karta hai:

$$\text{Loss} = -\log P(\text{target_class})$$

Kyunki array ka index integer hota hai ($0, 1, 2\ldots$), isiliye target `torch.long` (`int64`) hona mandatory hai. Agar float pass karenge toh `RuntimeError` aayega.

Solution 6: Manual Parameter Count Calculation

Formula: $\text{Params} = (\text{Inputs} \times \text{Outputs}) + \text{Outputs (biases)}$

1. Regression Network:

- Layer 1 (`4 -> 64`): $4 \times 64 + 64 = 256 + 64 = 320$
- Layer 2 (`64 -> 32`): $64 \times 32 + 32 = 2048 + 32 = 2,080$
- Layer 3 (`32 -> 1`): $32 \times 1 + 1 = 32 + 1 = 33$
- **Total**: $320 + 2080 + 33 = 2,433$ parameters.

2. Classification Network:

- Layer 1 (`34 -> 64`): $34 \times 64 + 64 = 2176 + 64 = 2,240$
- Layer 2 (`64 -> 64`): $64 \times 64 + 64 = 4096 + 64 = 4,160$
- Layer 3 (`64 -> 7`): $64 \times 7 + 7 = 448 + 7 = 455$
- **Total**: $2,240 + 4,160 + 455 = 6,855$ parameters.

Solution 7: Softmax aur `nn.CrossEntropyLoss` Ka Trap

PyTorch ka `nn.CrossEntropyLoss()` internally `LogSoftmax` + `NLLLoss` apply karta hai ek mathematically stable formula (**Log-Sum-Exp trick**) ke saath:

$$\log \sum e^{z_j} = c + \log \sum e^{z_j - c}, \quad c = \max(z_j)$$

Is trick se computer mein exponent values explode nahi hoti. Agar aapne model mein bhi `nn.Softmax()` laga diya, toh model probabilities emit karega. `CrossEntropyLoss` un probabilities par dobara `LogSoftmax` laga dega, jisse gradients shrink ho jayenge aur training tabah ho jayegi!

Solution 8: PyTorch Optimization Ke 4 Pillars

1. `optimizer.zero_grad()` : Purane gradients ko zero karta hai. Agar zero na karein, toh PyTorch gradients ko add karta rehta hai (`p.grad += new_grad`).
2. `outputs = model(inputs)` : Forward pass chala kar predictions nikalta hai.
3. `loss.backward()` : Backpropagation chala kar har weight ka gradient calculate karta hai.
4. `optimizer.step()` : Naye gradients ke mutabiq weights ko update karta hai ($w = w - \eta \cdot \nabla$).

- **Zero grad na karne par kya hoga?** Gradients har batch ke saath accumulate hokar bohot bade ho jayenge (Gradient Explosion), aur loss `NaN` ho jayega.

Solution 9: `model.state_dict()` vs Full Model Save Karna

- `state_dict` ek clean dictionary hoti hai jismein layer names aur unke numerical weight tensors store hote hain.
- `torch.save(model, path)` poore Python object ko pickle karta hai. Agar aap code refactor karenge ya kisi dusre computer par run karenge jahan class definition match nahi karegi, toh `pickle` crash ho jayega.
- Isiliye production mein hamesha `torch.save(model.state_dict(), "best_model.pt")` save kiya jata hai.

Solution 10: Model Checkpointing aur Overfitting Se Bachav

Training ke dauran training loss hamesha ghattata rehta hai. Lekin ek point ke baad validation loss badhne lagta hai (Overfitting). Agar hum aakhri epoch ka model rakhein, toh wo overfit model hoga. Checkpointing se hum us epoch ke weights save kar lete hain jahan validation loss minimum tha. `best_val_loss = float('inf')` isiliye rakhte hain taaki pehle epoch ka validation loss hamesha usse chhota ho aur pehla checkpoint save ho sake.

Solution 11: Evaluation Guardrails (`model.eval()` aur `torch.no_grad()`)

- `model.eval()` : Dropout ko band karta hai aur BatchNorm ko fixed running statistics par set karta hai. Lekin yeh gradients calculate hona band **nahi** karta.
- `with torch.no_grad():` PyTorch ko computational graph banane se rokta hai, jisse memory 70% bachti hai aur inference tez ho jata hai. Lekin yeh dropout ko band **nahi** karta.
- Isiliye evaluation ke time **dono ko ek saath** use karna padta hai:

```
model.eval()
with torch.no_grad():
    predictions = model(X_test_tensor)
```

Solution 12: Accuracy Calculation Mein Dynamic Batch Size (`yb.size(0)`)

Test dataset mein 180 samples hain. 32 ke hisaab se:

- Pehle 5 batches = $5 \times 32 = 160$ samples.
- Aakhri 6th batch = $180 - 160 = 20$ samples. Agar loop mein hardcoded `total += 32` likhenge, toh 6 batches ke baad total $6 \times 32 = 192$ ho jayega! Jabki asal samples 180 the. Isse accuracy galat evaluate hogi. `yb.size(0)` batch ke exact size (20) ko accurately count karta hai.

Solution 13: Regression Metrics Ka Interpretation

1. **RMSE**:

$$\text{RMSE} = \sqrt{17.81} \approx 4.22 \text{ MW}$$

   Iska matlab hai ki test data par model ka prediction average ± 4.22 Megawatts deviate karta hai. Power plant ki output range ($420 - 495$ MW) mein yeh sirf $\approx 0.9\%$ relative error hai, jo behad accurate hai.

2. $R^2 = 0.937$: Model plant ke electrical output ke total variance ka 93.7% **hissa successfully explain** kar raha hai.

Solution 14: Logits vs Probabilities Par Argmax Ki Equivalence

Softmax formula:

$$P(y = i|\mathbf{x}) = \frac{e^{z_i}}{\sum e^{z_j}}$$

Kyunki denominator sabhi classes ke liye same positive number hai, aur e^u function monotonically increasing hota hai, isiliye:

$$z_a > z_b \iff P(a) > P(b)$$

Iska matlab raw logits z par `torch.max(outputs, 1)` chalana aur probabilities par chalana bilkul same class index return karta hai. Isiliye inference ke time Softmax compute karne ki koi zaroorat nahi hoti.

Solution 15: PCA + ANN Dimensionality Reduction Challenge

```
from sklearn.decomposition import PCA
import torch
import torch.nn as nn

# 1. 95% variance preserve karne ke liye PCA fit karein
pca = PCA(n_components=0.95)
X_train_pca = pca.fit_transform(X_train_scaled)
X_test_pca = pca.transform(X_test_scaled)
k = X_train_pca.shape[1]

# 2. PyTorch Tensors
X_train_tensor = torch.tensor(X_train_pca, dtype=torch.float32)
```

```
X_test_tensor = torch.tensor(X_test_pca, dtype=torch.float32)

# 3. Model with k input features
class PCAModel(nn.Module):
    def __init__(self, in_features=k, num_classes=7):
        super(PCAModel, self).__init__()
        self.model = nn.Sequential(
            nn.Linear(in_features, 64),
            nn.ReLU(),
            nn.Linear(64, 64),
            nn.ReLU(),
            nn.Linear(64, num_classes)
        )
    def forward(self, x):
        return self.model(x)
```

Isse input features 34 se ghat kar k ho jate hain, parameters kam hote hain, aur model tez train hota hai.